

Ros Day01 hw1 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.09

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 Ros2 과제에서는 처음 접하는 Ros2 의 기본 개념을 이해하고, turtlesim 을 이용하여 원, 사각형, 삼각형을 그리는 과정에서 chatGPT 를 활용하였다.

처음에는 Ros 가 정확히 무엇인지, Node 와 Topic 이 어떤 역할을 하는지 이해하기 어려웠다. 특히 publisher, subscriber, message 와 같은 들어보지 못한 용어가 한꺼번에 등장하여 각각이 어떻게 연결되는지 이해하는데 어려움이 있었다.

ai 에게 ros 의 개념을 질문하면서 각각의 역할을 간단한 예시와 turtlesim 동작에 연결하여 이해하였다. 이후 직접 터미널에서 명령어를 실행하면서 실제 ros2 의 통신 과정을 확인하였다.

2. AI 사용 부분

2.1 ROS2 기본 개념 이해

처음에는 ROS 가 일반적인 운영체제인지, 로봇을 제어하는 프로그램인지 구분하기 어려웠다.

AI 에게 질문하여 Ros2 는 하나의 프로그램이 모든 기능을 수행하는 것이 아니라 여러개의 Node 로 기능을 나누고, 이 Node 들이 message 를 주고 받으면서 동작하는 구조라는 것을 이해할 수 있었다.

특히 다음과 같이 정리하여 이해하였다.

Node

→ 실행되는 하나의 프로그램

Message

→ Node 사이에서 주고받는 데이터

Topic

→ Message 가 전달되는 통신 채널

Publisher

→ Message 를 보내는 쪽

Subscriber

→ Message 를 받는 쪽

이건 쉽게 보면:

```
linear.x = 앞으로/뒤로 가는 속도  
angular.z = 좌우로 도는 속도
```

예를 들어:

```
linear.x = 2.0  
angular.z = 0.0
```

이면 **앞으로 직진**.

반대로:

```
linear.x = 0.0  
angular.z = 2.0
```

이면 **제자리에서 회전**.

네가 방향키 누를 때마다 `turtle_teleop_key` 가 이런 속도값을 메시지로 만들어서 `/turtle1/cmd_vel` 에 보내고, `turtlesim_node` 가 받아서 움직이는 거야. 자료에서도 teleop 노드는 속도값을 publish하고 turtlesim 노드는 그 토픽을 받아 움직인다고 설명돼 있어.

 eee624db-e300-4646-bc4f-7b3c76c...

지금 네가 실제로 확인한 전체 구조는 이거야.

```
방향키 입력  
↓  
turtle_teleop_key  -- Node / Publisher  
↓  
속도 Message  
↓  
/turtle1/cmd_vel  -- Topic  
↓  
turtlesim_node    -- Node / Subscriber  
↓  
거북이 움직임
```

그리고 turtlesim에 선이 막 그려진 것도 정상임. 네가 방향키로 계속 움직여서 거북이가 이동한 자취가 남은 거야.

이제 다음은 이걸 **그림으로 확인하는 rqt_graph** 해보자.

`echo` 켜둔 터미널은 `Ctrl+C` 로 끄고:

```
</> Bash  
rqt_graph
```

쳐봐. 그러면 `teleop_turtle` → `/turtle1/cmd_vel` → `turtlesim` 연결 관계가 GUI로 보일 거야. 이 키보드 오프라인이 아니니  eee624db-e300-4646-bc4f-7b3c76c...

+ ChatGPT에게 물어보세요

High



2.2 사각형과 삼각형의 회전 오차 해결

사각형을 만들기 위해 처음에는 빠른 각속도로 약 90 도씩 회전하도록 설정하였다. 하지만 같은 명령어를 반복하여 실행해도 회전 결과가 조금씩 달라졌으며, 사각형이 제대로 그려지지 않았다.

처음에는 `angular.z` 값을 조금씩 변경하며 맞추려고 하였지만 값이 조금이나 바뀌어도 회전결과와의 차이가 크게 나타났다.

따라서 최종적으로 회전 각속도를 0.2 까지 낮추어 오차를 최소화 하였다.

이거 뭐 값이 돌릴때마다 결과가 달라지는데 어떻게 해야하나

52s 동안 처리함 >

○○ 그러면 값 문제라기보다 방법 자체가 불안정한 거야. `timeout 1s ros2 topic pub ...` 는 “거북이를 정확히 1초 돌려라”가 아니라 ROS2 CLI 프로세스를 약 1초 뒤에 죽이는 방식이라, 시작/발견/메시지 발행 타이밍 때문에 매번 결과가 조금씩 달라질 수 있어. ROS2 공식 문서도 `ros2 topic pub` 의 실제 수신 주기는 시스템 자원이나 QoS 영향으로 설정한 rate와 정확히 같지 않을 수 있다고 설명해. [ROS Docs](#)

그래서 지금처럼 사각형 각도를 시간으로 맞추는 건 계속 흔들릴 가능성이 큼.

먼저 다른 Publisher가 섞이는지도 확인해보자. turtlesim만 켜놓고 새 터미널에서:

```
</> Bash
ros2 topic info /turtle1/cmd_vel -v
```

여기서 `Publisher count` 가 0이어야 정상적이야. `teleop` 이나 예전 `ros2 topic pub` 이 살아 있으면 여러 명이 동시에 속도 명령을 보내서 더 이상하게 움직일 수 있어.

그다음 사각형은 회전 속도를 확 낮춰서 직접 눈으로 90도에 맞추는 방식이 차라리 안정적이야.

회전:

```
</> Bash
ros2 topic pub -r 20 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, an
```

이러면 천천히 도니까 90도 근처에서 `ctrl+c` 하고 바로:

```
</> Bash
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, a
```

보내면 돼.

더 정확히 하고 싶으면 옆 터미널에서:

```
</> Bash
ros2 topic echo /turtle1/pose
```

를 켜놔. 여기서 `theta` 가 현재 각도야.

```
오른쪽 ≈ 0
위쪽 ≈ 1.57
왼쪽 ≈ 3.14 또는 -3.14
아래쪽 ≈ -1.57
```

+ | ChatGPT에게 물어보세요

High ▾



ROS2 뉴도터미널에서도 `/turtle1/pose` 를 `ros2 topic echo` 도 확인할 수 있다고 안내해.

3. 깨달은 점

이번 과제를 통해 Ros2 에서 깨달은 것은 Node 끼리 단순히 직접 연결되는 것이 아니라 Topic 을 이용하여 Message 를 주고 받는 구조라는 것이다.

또한 ros2 topic echo 와 rqt_graph 를 사용해 실제 메시지와 Node 사이의 연결구조를 확인하면서 publisher 와 subscriber 의 관계를 더 쉽게 이해할 수 있었다.

이번 과제를 통해 Ros2 의 node, topic, message, publisher, subscriber 개념을 알 수 있었고 직접 turtlesim 을 움직이며 이 개념들을 더욱 쉽게 이해할 수 있었던 것 같다.